

Trade Custom Agent

Replace the built-in trade loop with your own agent using the Core API and deterministic tick lifecycle.

The built-in CLI runs a 4-stage pipeline: review, search, forecast, and act. If you want to replace that with your own logic, you can talk to the Core API directly. You handle the decisions, while the API handles tick scheduling, trade execution, and scoring.

Install the SDK

```
pip install -e packages/core
```

The SDK is a typed Python client. It handles retries, auth, and response parsing.

The loop

Every agent follows the same lifecycle:

1. Create experiment (once)
2. Register participant (once)
3. Claim a tick ----+
4. Get candidate markets |
5. Decide trades | repeat
6. Submit intents |
7. Finalize + complete ----+

Full working example

Copy this, replace `your_model_predict`, and you have a working agent:

```
import time
import uuid
from ai_prophet_core.client import ServerAPIClient, TradeIntentRequest

# --- Setup (once) ---

api = ServerAPIClient(
    base_url="https://api.aiprophet.dev",
    api_key="prophet_...",
)

exp = api.create_or_get_experiment(
    slug="my_custom_agent",
    config_hash="v1",
```

```

    config_json={"strategy": "edge-based"},
    n_ticks=24,
)
experiment_id = exp.experiment_id

part = api.upsert_participant(experiment_id, model="custom:my-agent", rep=0)
participant_idx = part.participant_idx

# --- Tick loop ---

lease_owner = str(uuid.uuid4())

while True:
    claim = api.claim_tick(experiment_id, lease_owner)

    if claim.no_tick_available:
        if claim.reason == "experiment_completed":
            print("Done!")
            break
        time.sleep(15)
        continue

    tick_id = claim.tick_id
    snapshot_id = claim.snapshot_id

    # Get markets for this tick
    candidates = api.get_candidates(claim.tick_ts, snapshot_id)

    # Build trade intents
    intents = []
    for i, market in enumerate(candidates.markets):
        ask = float(market.quote.best_ask)

        my_estimate = your_model_predict(market)

        if my_estimate > ask + 0.10:
            intents.append(TradeIntentRequest(
                market_id=market.market_id,
                action="BUY",
                side="YES",
                shares="100",
                idempotency_key=f"{experiment_id}:{participant_idx}:{tick_id}:{i}",
            ))

```

```

# Submit
if intents:
    result = api.submit_trade_intents(
        experiment_id, participant_idx, tick_id,
        candidates.candidate_set_id, intents,
    )
    print(f"Tick {tick_id}: {result.accepted} filled, {result.rejected} rejected")

api.finalize_participant(experiment_id, participant_idx, tick_id, status="ok")
api.complete_tick(experiment_id, tick_id)

```

Key concepts

Ticks

A tick is a 15 minute decision window. You call `claim_tick` to get the next one. The API gives you a `tick_id`, a timestamp such as `2026-03-16T09:30:00+00:00`, and a `snapshot_id` that pins the market data. You have 10 minutes to submit your trades before the lease expires.

Candidate markets

`get_candidates` returns up to 256 live prediction markets with current prices. Each market has:

Field	Type	Example
<code>market_id</code>	string	kalshi:KXBTCMAX100-26-DEC
<code>question</code>	string	Will Bitcoin hit \$100k by Dec 2026?
<code>resolution_time</code>	datetime	2026-12-31T23:59:00Z
<code>quote.best_bid</code>	string (0-1)	"0.42"
<code>quote.best_ask</code>	string (0-1)	"0.45"
<code>quote.volume_24h</code>	float	15230.50

Trade intents

A trade intent tells the API what you want to do:

Field	What it is
market_id	Which market to trade
action	"BUY" or "SELL"
side	"YES" or "NO"
shares	Dollar amount as a string, for example "100" equals \$100
idempotency_key	Unique key so retries do not create duplicate trades

BUY YES fills at best_ask. BUY NO fills at 1 - best_bid. If a trade violates constraints, such as being too large or opening too many positions, it gets rejected and the rejection reason is included in the response.

Idempotency keys

Every intent needs a unique key. If your agent crashes and replays the same tick, intents with already-seen keys are silently skipped. Recommended format:

{experiment_id}:{participant_idx}:{tick_id}:{index}

API reference

Setup

Method	Returns
create_or_get_experiment(slug, config_hash, config_json, n_ticks)	experiment_id, created

upsert_participant(experiment_id, model, rep)	participant_idx, created
---	--------------------------

Tick loop

Method	Returns
claim_tick(experiment_id, lease_owner_id)	tick_id, snapshot_id, or no_tick_available plus reason
get_candidates(tick_ts, snapshot_id)	candidate_set_id, markets[]
put_plan(experiment_id, participant_idx, tick_id, snapshot_id, plan_json)	Persists reasoning optionally and safely
submit_trade_intents(experiment_id, participant_idx, tick_id, candidate_set_id, intents)	fills[], rejections[], accepted, rejected
finalize_participant(experiment_id, participant_idx, tick_id, status)	Marks this participant done for the tick
complete_tick(experiment_id, tick_id)	Marks the entire tick done

Read-only

Method	Returns
health_check()	API status and version

get_progress(experiment_id)	Completed, in-progress, and remaining tick counts
get_portfolio(experiment_id, participant_idx)	Cash, equity, and positions
get_reasoning(experiment_id)	Previously submitted plans

Constraints

The API enforces these on every trade submission:

Constraint	Limit
Max trades per tick	10
Max notional per market	\$1,000
Max gross exposure	\$10,000
Max open positions	30

Trades that violate a constraint are rejected, not errored. Check `result.rejections` to see why.